ZPR Policy Delegation

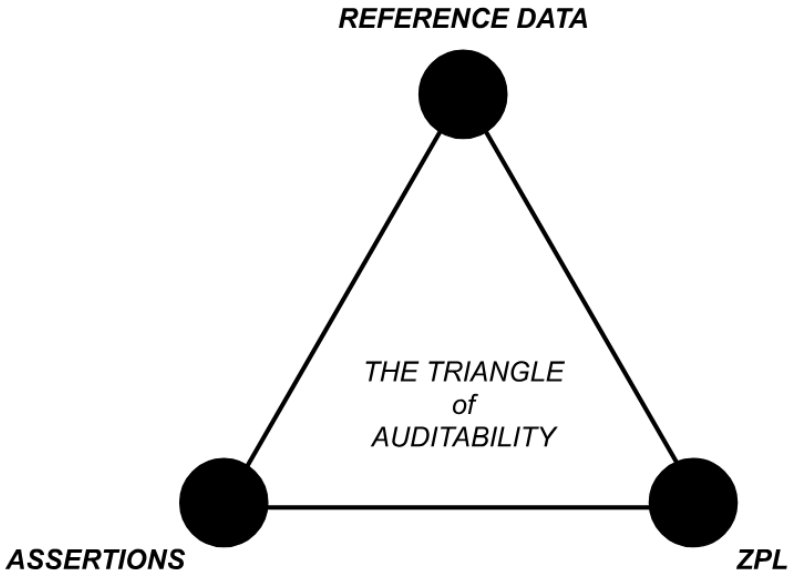
Mathias Kolehmainen, Michael Dubno
January 26, 2026

1. Introduction

In ZPR, policy delegation is the mechanism that allows a central authority to safely share control of network access policy with subordinate policy authors while still enforcing a coherent global security posture. As ZPR deployments grow in size and organizational complexity, delegation becomes necessary to distribute policy authoring without fragmenting control or weakening security guarantees.

In a non-trivial ZPR installation, policy delegation is only one part of the overall network security environment. In addition to policy, ZPR incorporates reference data from trusted services, and policy may be subject to assertions. Trusted service access and assertion creation have delegation mechanisms of their own that are outside the scope of policy delegation described here. The management of reference data, the authoring of assertions, and the authoring of ZPL are typically performed by different entities within an organization. In addition, to ensure a secure environment, all of these aspects of ZPR must be auditable.

The "Triangle of Auditability" diagram below illustrates that a complete ZPR environment involves three separately managed domains.



The Triangle of Auditability

The remainder of this paper focuses specifically on delegation support for the authoring of ZPL policies. It describes how delegated policy is constrained, verified, and enforced, and how reference data from trusted services is used safely within those constraints. While

assertions are a critical part of the overall ZPR security model, their delegation, authoring, and enforcement semantics are not defined here and are treated as future work.

2. Delegation Model and Hierarchy

In policy delegation, what is delegated is the ability to control access to services. The system guarantees that delegated policy is authentic and remains within the scope defined by the delegating administrator. The delegation act itself produces explicit evidence that the visa service can verify. This evidence, which we here call a "token", is part of the policy configuration.

Delegation always involves a hierarchy of administrators. At the top sits a root or high privilege administrator (A) who owns the global view of the network. Administrator A can delegate to many downstream administrators (B through Z). This hierarchy can be very shallow in practice; for example, a single top-level admin that delegates to many peers, or deeper if there are multiple layers of regional or functional admins.

The hierarchy has two important consequences:

1. When A delegates to B, A defines the set of services that fall under B's authority and sets the credentials with which B's policy will access trusted services. A always retains the ability to define policy for a delegated scope. An administrator never controls anything outside the scope defined by its parent, and the root administrator, who has no parent, can control everything.

2. When traffic is evaluated, the visa service matches policies in hierarchical order. Policies from higher level administrators are evaluated before policies from their delegates. This ordering ensures that the top-level policy can always impose global constraints, while still allowing delegated admins to define more specific behavior within their assigned slice.

A crucial invariant is that "never allow" rules are enforced everywhere in the hierarchy. If a delegated admin B adds a never allow rule that conflicts with an allow rule written by A, the never allow must take precedence. This is a deliberate design choice in favor of safety: it is better to deny incorrectly than to allow incorrectly. Operationally, such a conflict is a human coordination problem and should trigger a conversation between A and B, but the system behavior is deterministic and conservative.

3. Delegation Attributes

Each delegated policy is tied to a specific domain of control through delegation attributes. When admin A delegates to admin B, B is given a policy configuration that contains:

- A proof of delegation token that attests cryptographically that the policy for B was legitimately delegated from A.

- One or more delegation attributes that must apply to every service in B's policy.

A delegation attribute might be a key/value pair such as `dept:marketing`, a tag such as `marketing`, or a requirement that a particular attribute key simply be present. These attributes carve out a well defined and unambiguous subset of the ZPRnet that B is allowed to control.

For example, if `dept:marketing` is the delegated attribute for the marketing department, then only services that are actually part of marketing should ever have that attribute. Moreover, those services must not simultaneously hold other delegated attributes that belong to different administrative domains, since that would break the clean separation of concerns. This restriction keeps the attribute space partitioned into disjoint delegated regions, which simplifies reasoning about policy and prevents accidental cross domain control.

4. **Compiler Responsibilities**

The policy compiler is the first enforcement point for delegation. Given a delegated policy configuration, it is responsible for:

- Injecting the correct delegation attributes into every service stanza in the policy.
- Ensuring that the policy only uses attributes that are allowed for that delegated author.

The compiler determines the set of allowed attributes from the trusted services. That set may then be further reduced by the policy configuration itself. For example, the configuration can explicitly list attributes that are permitted or denied for use by the delegated policy. These constraints are enforced statically at compile time. This design prevents a delegated administrator from accidentally or intentionally authoring policy that escapes their delegated scope.

5. **Visa Service Behavior**

In a non delegated environment, a visa service runs a single policy and makes decisions based purely on that policy. In a delegated environment, the visa service runs many policies at once, one for each delegated administrator in the hierarchy.

To support this, the visa service:

- Confirms the proof of delegation token associated with each policy, so malformed or forged delegations are rejected.
- Orders the policies according to their place in the delegation hierarchy so that evaluation always begins with the highest ancestor.
- Enforces that each delegation attribute is only assigned to a single policy and that each policy uses only its own delegation attributes, taking the hierarchy into account.

Each policy maintains its own attribute cache for the trusted services it uses. This keeps attributes in delegated zones isolated from one another and reduces the impact of

configuration mistakes. At the same time, the visa service uses global credentials when talking to the authentication service because identity verification is a shared concern, not scoped per delegation.

To grant a visa for a specific request, the visa service first identifies the policy that has scope for the service along with all its parents. Then the visa service checks the policies in hierarchical order. Within a single hierarchical level there is no specified order: all peers at that level are equivalent. Access is granted when the first policy permits it, searching in hierarchical order. Access is denied when any policy in the hierarchy matches on a "never-allow" rule, or no policy allows the traffic. Access to a service is always controlled by either:

- The policy that was delegated for the attributes that select that service, or
- A policy higher in the same chain of delegation.

This rule ensures that no service is left in a gray area where multiple unrelated policies appear to control it. Every access decision is owned either by the local delegated admin or by one of its ancestors.

6. **Trusted Services and Attribute Discipline**

Trusted services play a key role in enforcement by strictly controlling attribute distribution. They must:

- Only return delegated attributes to the actors authorized to receive them.
- Avoid exposing conflicting attribute sets that would place a service under two different delegated domains.

Trusted services that back attributes may also support different views of data based on credentials. This may be a whole separate delegated access hierarchy not under the control of ZPR. ZPR, as a client of trusted services is, of course, bound by whatever hierarchy is configured behind the scenes. For example, Active Directory may be configured such that users with different credentials are able to query for and view different attributes. Service access credentials are part of delegated policy configuration so an administrator can take advantage of this to sandbox delegated administrators. Since the policy configuration requires that all attributes are declared, the ZPR compiler can detect if attributes listed in configuration are not permitted when using a service credential.

It is important to note that this carving up of the ZPR network only works if attributes are correctly managed in the underlying systems. ZPR relies on these services to return accurate attribute sets and to avoid exposing attributes that would give a delegated administrator influence outside its authorized scope. If attribute assignment in the underlying systems is not disciplined, ZPR cannot correct the mistake; instead, the model assumes that attribute management is accurate and auditable.

When this discipline is followed, the delegation model in ZPR achieves three goals simultaneously:

1. It lets large organizations distribute policy authoring to many admins without losing central control.

2. It keeps the semantics of delegation aligned with the semantics of normal access control, by expressing both as attributes and policy.

3. It provides clear, auditable boundaries for administrative authority. The system behaves correctly as long as attribute sources remain consistent and well managed. Auditing the delegated attribute space is straightforward, and misconfigurations can be detected through verification of delegation tokens, compiler checks, and visa service validation.

7. Future Work

1. Service discoverability is an unresolved design issue for ZPR but will be closely tied to delegation. The ability to find a service, not just access it, should be governed by policy rules.

2. Originally conceived as a part of ZPL, assertions are also useful as their own set of stand-alone rules. The “Triangle of Auditability” vertex labelled “assertions” is referring to this stand-alone concept of assertions. For example, there may be assertions written that constrain the reference data or any of the ZPL content. Who can write assertions and about what – this will involve delegation and control just like policy and reference data.

8. Revision History

1. Revision as of Jan 26, 2026, ported out of Google docs.